

9. Python Implementation

```

from typing import List

class Backtracking:
    # =====
    # 1. Generate All Permutations
    # =====
    def permute(self, nums: List[int]) -> List[List[int]]:
        result = []
        current = []
        used = [False] * len(nums)

        def backtrack():
            if len(current) == len(nums):
                result.append(current[:])
                return

            for i in range(len(nums)):
                if used[i]:
                    continue
                used[i] = True
                current.append(nums[i])
                backtrack()
                current.pop()
                used[i] = False

        backtrack()
        return result

    # =====
    # 2. Generate All Subsets
    # =====
    def subsets(self, nums: List[int]) -> List[List[int]]:
        result = []
        current = []

        def backtrack(start: int):
            result.append(current[:])
            for i in range(start, len(nums)):
                current.append(nums[i])
                backtrack(i + 1)
                current.pop()

        backtrack(0)
        return result

    # =====
    # 3. N-Queens
    # =====
    def solve_n_queens(self, n: int) -> List[List[str]]:

```

```

result = []
board = [['.'] * n for _ in range(n)]
cols = set()
diag1 = set() # row - col
diag2 = set() # row + col

def backtrack(row: int):
    if row == n:
        result.append(''.join(r) for r in board)
        return

    for c in range(n):
        d1 = row - c
        d2 = row + c
        if c in cols or d1 in diag1 or d2 in diag2:
            continue

        board[row][c] = 'Q'
        cols.add(c)
        diag1.add(d1)
        diag2.add(d2)
        backtrack(row + 1)
        board[row][c] = '.'
        cols.remove(c)
        diag1.remove(d1)
        diag2.remove(d2)

backtrack(0)
return result

# =====
# 4. Combination Sum
# =====
def combination_sum(self, candidates: List[int], target: int) ->
List[List[int]]:
    result = []
    current = []

    def backtrack(start: int, remaining: int):
        if remaining == 0:
            result.append(current[:])
            return
        if remaining < 0:
            return

        for i in range(start, len(candidates)):
            current.append(candidates[i])
            backtrack(i, remaining - candidates[i])
            current.pop()

    backtrack(0, target)
    return result

```

```
# Example usage
if __name__ == "__main__":
    bt = Backtracking()

    # Permutations
    print("Permutations of [1,2,3]:")
    for p in bt.permute([1, 2, 3]):
        print(f" {p}")

    # Subsets
    subs = bt.subsets([1, 2, 3])
    print(f"Subsets (count={len(subs)}): {subs}")

    # N-Queens
    solutions = bt.solve_n_queens(4)
    print(f"N-Queens (4) solutions: {len(solutions)}")
    for board in solutions:
        for row in board:
            print(f" {row}")
        print()

    # Combination Sum
    print("Combination Sum target=7 from [2,3,6,7]:")
    for c in bt.combination_sum([2, 3, 6, 7], 7):
        print(f" {c}")
```

10. Code Explanation

Permutations:

- `used[i]` tracks whether `nums[i]` is already in the current permutation.
- At each step, we try all unused elements.
- When `current.size() == n`, we have a complete permutation.
- The "choose, explore, unchoose" pattern: mark used → push → recurse → pop → unmark.

Subsets:

- Uses `start` index to avoid generating duplicate subsets (prevents [1,2] and [2,1]).
- At each step, we add the current subset to result (including empty).
- We only consider elements from `start` onward, ensuring each element is added at most once.

N-Queens:

- Track columns, main diagonals (`row - col`), and anti-diagonals (`row + col`) where queens are placed.
- For each row, try each column. If the position is not attacked by any existing queen, place a queen.
- `row - col` is constant for a diagonal (top-left to bottom-right).
- `row + col` is constant for an anti-diagonal (top-right to bottom-left).

Combination Sum:

- Uses `start` index, but allows reusing the same element by passing `i` (not `i+1`) to the recursive call.
- Sort candidates first to enable pruning (skip when remaining < 0).

11. Complexity Analysis

Problem	Time	Space	Notes
Permutations (n)	$O(n \times n!)$	$O(n)$	$n!$ permutations, each $O(n)$ to copy
Subsets (n)	$O(n \times 2^n)$	$O(n)$	2^n subsets
N-Queens	$O(n!)$	$O(n^2)$	Upper bound, heavily pruned
Combination Sum	$O(2^{(\text{target}/\text{min})})$	$O(\text{target}/\text{min})$	Depends on candidates
Sudoku	$O(9^{(\text{empty})})$	$O(81)$	Heavily pruned in practice

Key insight: Backtracking is exponential in worst case. Pruning reduces the search space significantly for practical problems.

12. Common Patterns

Pattern 1: Permutations (Order Matters)

Identify: "All permutations", "arrangements". **Approach:** `used` boolean array, try all unused elements.

Example: LeetCode 46 — Permutations.

Pattern 2: Combinations / Subsets (Order Doesn't Matter)

Identify: "All subsets", "all combinations", "choose k elements". **Approach:** `start` index, only consider elements from `start` onward. **Example:** LeetCode 78 — Subsets, LeetCode 77 — Combinations.

Pattern 3: Constraint Satisfaction (N-Queens, Sudoku)

Identify: "Place n items such that no two attack each other", "solve puzzle". **Approach:** Validate constraints after each placement, prune if violated. **Examples:** LeetCode 51 — N-Queens, LeetCode 37 — Sudoku Solver.

Pattern 4: Combination Sum (Unbounded Knapsack)

Identify: "Combinations that sum to target", "can reuse elements". **Approach:** Sort, use `start` but allow reusing (pass `i` not `i+1`). **Example:** LeetCode 39 — Combination Sum.

Pattern 5: Subsets with Duplicates

Identify: "Array has duplicates, return all unique subsets." **Approach:** Sort input, skip duplicates at same recursion level: `if (i > start && nums[i] == nums[i-1]) continue`. **Example:** LeetCode 90 — Subsets II.

Pattern 6: Palindrome Partitioning

Identify: "Partition string into palindromes." **Approach:** At each step, try all prefixes that are palindromes, recurse on remaining string. **Example:** LeetCode 131 — Palindrome Partitioning.

13. Common Mistakes

- **Not making a copy of the current state:** In Python, `result.append(current)` appends a reference. Use `current[:]` or `list(current)`.
- **Incorrect pruning:** Not checking all constraints, or checking too late (should check before recursing).
- **Missing start index:** Leads to duplicate permutations (elements in different orders).
- **Wrong duplicate handling:** For subsets with duplicates, only skip at the same level, not across levels.
- **Modifying global state without restoring:** Always undo modifications after recursive call.
- **Stack overflow:** Deep recursion (like for very long strings in palindrome partitioning).
- **Not sorting for combination sum:** Sorting helps prune early when `remaining < candidate[i]`.

14. Edge Cases

Case	Expected Behavior
Empty input	Return [[]] for subsets, [] for permutations
Single element	Permutations: [[1]], Subsets: [], [1]
All same values (with duplicates)	Handle duplicates with sorting + skipping
Target = 0 (combination sum)	Return [[]] (empty combination)
N = 1 (N-Queens)	One solution: [["Q"]]
N = 2 or 3 (N-Queens)	No solutions (return empty)
No solution exists	Return empty
Large n (n > 12 for perm)	Exponential explosion, may not finish

15. Variations

Branch and Bound

Backtracking with optimization (find best solution, not all). Maintain best solution found so far. Prune if current partial solution can't beat the best.

Backtracking with Bitmask

Use integer bitmask instead of boolean array for used tracking. Faster and more memory efficient.

```
// Permutations with bitmask
void backtrack(vector<int>& current, int mask, vector<int>& nums) {
    if (current.size() == nums.size()) { /* save */ return; }
    for (int i = 0; i < nums.size(); i++) {
        if (mask & (1 << i)) continue;
        current.push_back(nums[i]);
        backtrack(current, mask | (1 << i), nums);
        current.pop_back();
    }
}
```

Iterative Backtracking

Use explicit stack instead of recursion. More complex but avoids stack overflow.

Backtracking with Heuristics

For problems like Sudoku, choose the cell with the fewest possibilities first (MRV — Minimum Remaining Values heuristic).

16. Related Algorithms/Data Structures

Algorithm	Connection
DFS	Backtracking IS DFS on the state space tree.
BFS	BFS can also explore state space (finds shortest sequence of moves).
Dynamic Programming	For problems with overlapping subproblems, DP is better (memoization avoids recomputation).
Recursion	Backtracking is inherently recursive.

Backtracking vs DP:

- Backtracking: Explores all possibilities, no overlapping subproblem detection.
- DP: Recognizes overlapping subproblems, stores results to avoid recomputation.
- Use DP when the problem has optimal substructure and overlapping subproblems.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Subsets	LeetCode 78	All subsets	Medium
Permutations	LeetCode 46	All permutations	Medium

Medium

Problem	Platform	Pattern	Difficulty
Combination Sum	LeetCode 39	Combinations with sum	Medium
Subsets II	LeetCode 90	Subsets with duplicates	Medium
N-Queens	LeetCode 51	Constraint satisfaction	Hard

Hard

Problem	Platform	Pattern	Difficulty
Sudoku Solver	LeetCode 37	Constraint satisfaction	Hard

Problem	Platform	Pattern	Difficulty
N-Queens II	LeetCode 52	Count solutions	Hard
Word Search II	LeetCode 212	Backtracking with Trie	Hard

18. Interview Explanation

"Backtracking is a brute-force technique that incrementally builds candidates and abandons them when they can't lead to a valid solution. The core pattern is 'choose, explore, unchoose' — we make a choice, recursively explore further, then undo the choice. I use it for generating all permutations, combinations, and constraint satisfaction problems like N-Queens. The key to making backtracking efficient is pruning — cutting off invalid branches early. The time complexity is typically exponential, but good pruning makes it practical for moderate input sizes. In practice, I use a 'start' index for combinations (to avoid duplicates), a 'used' array for permutations, and constraint checks for problems like N-Queens."

19. Revision Notes

- **Core pattern:** Choose → Explore → Unchoose
- **Permutations:** `used[i]` array, try all unused.
- **Combinations:** `start` index, only try from `start` onward.
- **Pruning:** Check validity before recursing, not after.
- **Duplicates:** Sort + skip `if (i > start && nums[i] == nums[i-1])`.
- **Copy:** Always copy the current state when saving.
- **Complexity:** $O(n \times n!)$ perm, $O(2^n)$ subsets, $O(n!)$ N-Queens.
- **Common trap:** Not restoring state, not copying result, wrong `start` index.

20. Final Cheat Sheet

BACKTRACKING – CHEAT SHEET	
WHEN TO USE:	All permutations/combinations/subsets, constraint satisfaction
PATTERN:	<pre>void bt() { if(done) save; for(choice in choices) if(isValid) { make; bt(); undo; } } </pre>
PERMUTATIONS:	<code>used[]</code> array
COMBINATIONS:	<code>start</code> index
DUPLICATES:	sort + skip same level
PRUNING:	Check constraints early
COMPLEXITY:	$O(n!)$ to $O(2^n)$
COMMON USE:	N-Queens, Sudoku, Subsets, Perms
TRAP:	Don't restore state, don't copy result, wrong index
RELATED:	DFS, DP (for overlapping subs)

13. DP (DYNAMIC PROGRAMMING)

1. Overview

Dynamic Programming (DP) is a technique for solving problems with **overlapping subproblems** and **optimal substructure** by breaking them into smaller subproblems, solving each once, and storing the results for reuse. DP avoids the exponential blowup of naive recursion by trading space for time.

2. Intuition

Core idea: If you have to compute the same thing multiple times, compute it once, remember the answer, and reuse it.

Analogy: You're climbing a staircase with n steps. You can take 1 or 2 steps at a time. How many ways to reach the top? The number of ways to reach step k is $\text{ways}(k-1) + \text{ways}(k-2)$. If you compute this recursively, you'll recompute the same values many times. With DP, you compute $\text{ways}(1)$, $\text{ways}(2)$, ..., $\text{ways}(n)$ once each.

Step-by-step reasoning:

1. Identify that a problem can be broken into smaller subproblems.
2. Define a recurrence relation (how larger solutions depend on smaller ones).
3. Solve subproblems from smallest to largest (bottom-up) or with memoization (top-down).
4. The answer is the final subproblem value.

Why it works: By storing subproblem results, we avoid recomputing them. The total work becomes $O(\text{number of subproblems} \times \text{work per subproblem})$.

3. When to Use It

- **Counting problems:** "Number of ways to..."
- **Optimization problems:** "Maximum/minimum value to..."
- **Decision problems:** "Is it possible to..."
- Problems with **overlapping subproblems** (same subproblem appears multiple times).
- Problems with **optimal substructure** (optimal solution built from optimal solutions of subproblems).
- **Classic DP problems:** Fibonacci, knapsack, LCS, LIS, edit distance, matrix chain multiplication.

Common trigger phrases:

- "maximum/minimum"
- "number of ways"
- "longest" / "shortest"
- "subsequence"
- "subset"
- "knapsack"
- "optimal"
- "overlapping"

4. When Not to Use It

- **No overlapping subproblems:** Divide and conquer (merge sort) is better.
- **Greedy works:** Simpler and faster.
- **Small constraints:** Brute force or recursion is fine.
- **Not optimal substructure:** The problem doesn't build from subproblems.
- **State space is too large:** If the DP table is too big for memory, consider greedy or approximation.

5. Core Concepts

5.1 Memoization (Top-Down)

Recursive approach with caching. Start from the original problem, recursively compute subproblems, store results.

```
int fib(int n, vector<int>& memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib(n-1, memo) + fib(n-2, memo);
}
```

Pros: Only computes needed subproblems, intuitive for recursive thinkers. **Cons:** Recursion overhead, risk of stack overflow.

5.2 Tabulation (Bottom-Up)

Iterative approach. Compute subproblems from smallest to largest, filling a table.

```
vector<int> dp(n+1);
dp[0] = 0; dp[1] = 1;
for (int i = 2; i <= n; i++) {
    dp[i] = dp[i-1] + dp[i-2];
}
return dp[n];
```

Pros: No recursion overhead, easier to optimize space, faster. **Cons:** Computes all subproblems even if not needed.

5.3 State & Transition

- **State:** What does $dp[i][j]$ represent? (e.g., $dp[i][j]$ = LCS of first i chars of string A and first j chars of string B)
- **Transition:** How does $dp[i][j]$ relate to smaller subproblems? (e.g., $dp[i][j] = \max(dp[i-1][j], dp[i][j-1]) + 1$)
- **Base case:** $dp[0][j] = 0$, $dp[i][0] = 0$

5.4 DP Patterns (Common)

- **1D DP:** $dp[i]$ depends on $dp[i-1]$, $dp[i-2]$, ...

- **2D DP:** $dp[i][j]$ depends on $dp[i-1][j]$, $dp[i][j-1]$, $dp[i-1][j-1]$
- **Knapsack:** $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$
- **Interval DP:** $dp[i][j]$ depends on $dp[i][k] + dp[k+1][j]$ for k in $[i, j]$
- **Tree DP:** $dp[node]$ depends on $dp[child]$ values

5.5 Space Optimization

If $dp[i]$ only depends on $dp[i-1]$, we can use rolling variables instead of an entire array.

```
// Fibonacci with O(1) space
int a = 0, b = 1;
for (int i = 2; i <= n; i++) {
    int c = a + b;
    a = b;
    b = c;
}
return b;
```

6. Step-by-Step Algorithm

Fibonacci (Bottom-Up):

1. Create dp array of size $n+1$.
2. Set $dp[0] = 0$, $dp[1] = 1$.
3. For $i = 2$ to n : $dp[i] = dp[i-1] + dp[i-2]$.
4. Return $dp[n]$.

0/1 Knapsack (Maximum value with weight capacity W):

1. Create dp table of size $(n+1) \times (W+1)$, initialized to 0.
2. For $i = 1$ to n (items):
 - For $w = 1$ to W (capacity):
 - If $weight[i-1] \leq w$: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - weight[i-1]] + value[i-1])$.
 - Else: $dp[i][w] = dp[i-1][w]$.
3. Return $dp[n][W]$.

Longest Common Subsequence (LCS):

1. Create dp table of size $(n+1) \times (m+1)$, initialized to 0.
2. For $i = 1$ to n :
 - For $j = 1$ to m :
 - If $text1[i-1] == text2[j-1]$: $dp[i][j] = dp[i-1][j-1] + 1$.
 - Else: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.
3. Return $dp[n][m]$.

7. Dry Run

Problem: Fibonacci $F(6)$

Bottom-Up:

i	dp[i] = dp[i-1] + dp[i-2]	Result
0	0	0
1	1	1
2	1 + 0	1
3	1 + 1	2
4	2 + 1	3
5	3 + 2	5
6	5 + 3	8

Result: F(6) = 8**Problem:** 0/1 Knapsack — Items: [(value=60, weight=5), (50, 3), (70, 4), (30, 2)], W = 5**DP Table (rows = items, cols = capacity):**

Item \ W	0	1	2	3	4	5
0 (none)	0	0	0	0	0	0
1 (5,60)	0	0	0	0	0	60
2 (3,50)	0	0	0	50	50	60
3 (4,70)	0	0	0	50	70	70
4 (2,30)	0	0	30	50	70	80

Result: Max value = 80 (items 2 and 4: weight 3+2=5, value 50+30=80)

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// =====
// 1. Fibonacci (Bottom-Up with space optimization)
// =====
int fibonacci(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
}

```

```

    return b;
}

// =====
// 2. 0/1 Knapsack
// =====
int knapsack01(const vector<int>& weights, const vector<int>& values, int W) {
    int n = weights.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {
            if (weights[i-1] <= w) {
                dp[i][w] = max(dp[i-1][w],
                               dp[i-1][w - weights[i-1]] + values[i-1]);
            } else {
                dp[i][w] = dp[i-1][w];
            }
        }
    }
    return dp[n][W];
}

// Space-optimized version (1D array)
int knapsack01Optimized(const vector<int>& weights, const vector<int>& values, int W) {
    int n = weights.size();
    vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; i++) {
        for (int w = W; w >= weights[i]; w--) {
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }
    return dp[W];
}

// =====
// 3. Longest Common Subsequence (LCS)
// =====
int longestCommonSubsequence(const string& text1, const string& text2) {
    int n = text1.size(), m = text2.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (text1[i-1] == text2[j-1]) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }
    return dp[n][m];
}

```

```

}

// =====
// 4. Longest Increasing Subsequence (LIS)
// =====
int lengthOfLIS(vector<int>& nums) {
    int n = nums.size();
    vector<int> dp(n, 1);
    int maxLen = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
        maxLen = max(maxLen, dp[i]);
    }
    return maxLen;
}

// =====
// 5. Coin Change (Minimum coins)
// =====
int coinChange(const vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i && dp[i - coin] != INT_MAX) {
                dp[i] = min(dp[i], dp[i - coin] + 1);
            }
        }
    }
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}

// =====
// 6. Edit Distance (Levenshtein Distance)
// =====
int editDistance(const string& word1, const string& word2) {
    int n = word1.size(), m = word2.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (word1[i-1] == word2[j-1]) {
                dp[i][j] = dp[i-1][j-1];
            } else {
                dp[i][j] = 1 + min({dp[i-1][j], // delete

```

```

        dp[i][j-1],    // insert
        dp[i-1][j-1]}); // replace
    }
}
}
return dp[n][m];
}

// Example usage
int main() {
    cout << "Fibonacci(6): " << fibonacci(6) << "\n"; // 8

    vector<int> weights = {5, 3, 4, 2};
    vector<int> values = {60, 50, 70, 30};
    cout << "Knapsack(5): " << knapsack01(weights, values, 5) << "\n"; // 80
    cout << "Knapsack Opt(5): " << knapsack01Optimized(weights, values, 5) <<
"\n"; // 80

    cout << "LCS: " << longestCommonSubsequence("abcde", "ace") << "\n"; // 3
("ace")

    vector<int> nums = {10, 9, 2, 5, 3, 7, 101, 18};
    cout << "LIS: " << lengthOfLIS(nums) << "\n"; // 4 (2,5,7,101)

    cout << "Coin Change(11): " << coinChange({1,2,5}, 11) << "\n"; // 3 (5+5+1)

    cout << "Edit Distance: " << editDistance("horse", "ros") << "\n"; // 3

    return 0;
}

```

9. Python Implementation

```

from typing import List

def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

def knapsack01(weights: List[int], values: List[int], W: int) -> int:
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, W + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(dp[i - 1][w],

```

```

        dp[i - 1][w - weights[i - 1]] + values[i - 1])
    else:
        dp[i][w] = dp[i - 1][w]
    return dp[n][W]

def knapsack01_optimized(weights: List[int], values: List[int], W: int) -> int:
    n = len(weights)
    dp = [0] * (W + 1)
    for i in range(n):
        for w in range(W, weights[i] - 1, -1):
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
    return dp[W]

def longest_common_subsequence(text1: str, text2: str) -> int:
    n, m = len(text1), len(text2)
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[n][m]

def length_of_lis(nums: List[int]) -> int:
    n = len(nums)
    dp = [1] * n
    max_len = 1

    for i in range(1, n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
        max_len = max(max_len, dp[i])
    return max_len

def coin_change(coins: List[int], amount: int) -> int:
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i:
                dp[i] = min(dp[i], dp[i - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

def edit_distance(word1: str, word2: str) -> int:
    n, m = len(word1), len(word2)
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):

```

```

    dp[0][j] = j

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
    return dp[n][m]

# Example usage
if __name__ == "__main__":
    print(f"Fibonacci(6): {fibonacci(6)}") # 8
    print(f"Knapsack(5): {knapsack01([5,3,4,2], [60,50,70,30], 5)}") # 80
    print(f"LCS: {longest_common_subsequence('abcde', 'ace')}") # 3
    print(f"LIS: {length_of_lis([10,9,2,5,3,7,101,18])}") # 4
    print(f"Coin Change(11): {coin_change([1,2,5], 11)}") # 3
    print(f>Edit Distance: {edit_distance('horse', 'ros')}") # 3

```

10. Code Explanation

Fibonacci (Space optimized):

- Only store last two values: $a = dp[i-2]$, $b = dp[i-1]$.
- Each iteration: $c = a + b$, shift $a = b$, $b = c$.

0/1 Knapsack:

- $dp[i][w]$ = max value using first i items with capacity w .
- Two choices for item i : skip it ($dp[i-1][w]$) or take it ($dp[i-1][w-w_i] + v_i$).
- The optimized version uses a 1D array, iterating w backwards to avoid using the same item twice.

LCS:

- $dp[i][j]$ = LCS of first i chars of text1 and first j chars of text2.
- If chars match: extend the LCS by 1 ($dp[i-1][j-1] + 1$).
- If not: take max of skipping one char from either string.

LIS ($O(n^2)$):

- $dp[i]$ = length of LIS ending at index i (including $nums[i]$).
- For each i , check all $j < i$: if $nums[j] < nums[i]$, extend LIS ending at j .
- $O(n \log n)$ version exists using patience sorting (binary search on tails array).

Coin Change:

- $dp[i]$ = minimum coins to make amount i .
- For each coin, try using it: $dp[i - coin] + 1$.
- Initialize $dp[0] = 0$, rest to INF.

Edit Distance:

- $dp[i][j]$ = min edits to convert first i chars of word1 to first j chars of word2.
- Three operations: delete ($dp[i-1][j] + 1$), insert ($dp[i][j-1] + 1$), replace ($dp[i-1][j-1] + 1$).
- If chars match, no operation needed: $dp[i-1][j-1]$.

11. Complexity Analysis

Problem	Time	Space	Space Optimized
Fibonacci	$O(n)$	$O(1)$	—
0/1 Knapsack	$O(nW)$	$O(nW)$	$O(W)$
LCS	$O(nm)$	$O(nm)$	$O(\min(n,m))$
LIS ($O(n^2)$)	$O(n^2)$	$O(n)$	—
LIS ($O(n \log n)$)	$O(n \log n)$	$O(n)$	—
Coin Change	$O(n \times \text{amount})$	$O(\text{amount})$	—
Edit Distance	$O(nm)$	$O(nm)$	$O(\min(n,m))$
Matrix Chain	$O(n^3)$	$O(n^2)$	—

12. Common Patterns

Pattern 1: 1D DP — Fibonacci-like

Identify: $dp[i]$ depends on $dp[i-1]$, $dp[i-2]$, etc. **Examples:** Climbing stairs, house robber, decode ways.

Pattern 2: Knapsack (0/1)

Identify: "Choose subset with max value under weight limit." Each item at most once. **Approach:** 2D or 1D (reverse loop) DP. **Examples:** LeetCode 416 — Partition Equal Subset Sum, LeetCode 494 — Target Sum.

Pattern 3: Unbounded Knapsack

Identify: "Choose items to make a sum/value", items can be reused. **Approach:** 1D DP, forward loop (unlike 0/1 knapsack reverse loop). **Examples:** LeetCode 322 — Coin Change, LeetCode 518 — Coin Change II.

Pattern 4: Longest Common Subsequence

Identify: "Longest common subsequence/substring." **Approach:** 2D DP, match $\rightarrow$ diag+1, no match $\rightarrow$ max of left/up. **Example:** LeetCode 1143 — Longest Common Subsequence.

Pattern 5: Longest Increasing Subsequence

Identify: "Longest increasing subsequence." **Approach:** $O(n^2)$ DP or $O(n \log n)$ with patience sorting. **Example:** LeetCode 300 — Longest Increasing Subsequence.

Pattern 6: Edit Distance / String Alignment

Identify: "Minimum operations to convert string A to B." **Approach:** 2D DP with delete/insert/replace. **Example:** LeetCode 72 — Edit Distance.

Pattern 7: Interval DP

Identify: "Optimal way to..." on a range. $dp[i][j]$ depends on $dp[i][k] + dp[k+1][j]$. **Examples:** Matrix chain multiplication, burst balloons, palindrome partitioning II.

Pattern 8: Grid DP (Unique Paths)

Identify: "Number of ways to reach bottom-right from top-left." **Approach:** $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ (or similar). **Examples:** LeetCode 62 — Unique Paths, LeetCode 64 — Minimum Path Sum.

Pattern 9: DP on Trees

Identify: "Maximum/minimum something in a binary tree." **Approach:** Postorder DFS, combine child results. **Examples:** LeetCode 337 — House Robber III, LeetCode 124 — Binary Tree Maximum Path Sum.

13. Common Mistakes

- **Wrong base case:** $dp[0]$ or $dp[0][*]$ initialization is critical.
- **Index out of bounds:** Using $dp[i-1]$ when $i = 0$.
- **Integer overflow:** Use long long for large values.
- **Wrong iteration direction in space-optimized knapsack:** 0/1 knapsack needs reverse loop (w from W to $weight[i]$). Unbounded knapsack uses forward loop.
- **Not resetting dp for INF:** Need to initialize with large value and check before using.
- **Confusing subsequence vs substring:** Subsequence (non-contiguous, LCS) vs substring (contiguous, need different DP).
- **Forgetting to handle the case where no solution exists.**
- **Incorrect recurrence:** Verify with small examples.

14. Edge Cases

Case	Expected Behavior
Empty input ($n=0$)	Return 0 or base value
Single element	Return that element or 1 based on problem
All same values	LIS = n , LCS depends on strings
Amount = 0 (coin change)	0 coins needed
Impossible amount	Return -1
$W = 0$ (knapsack)	Return 0
Large values	Use long long

15. Variations

Digit DP

DP on digit positions for counting numbers with certain properties (digits, sum, divisibility) in a range.

DP with Bitmask (DP over subsets)

$dp[mask]$ represents the state of a subset. Used for traveling salesman, matching problems.

DP with Convex Hull Trick (CHT)

For DP with transitions of the form $dp[i] = \min/\max(a[j] \times x[i] + b[j])$. Optimizes $O(n^2)$ to $O(n \log n)$.

DP with Divide and Conquer Optimization

For DP with $dp[i][j] = \min(dp[i-1][k] + C(k, j))$ where the optimal k is monotonic.

16. Related Algorithms/Data Structures

Algorithm	Connection
Greedy	Simpler but doesn't always work. DP when greedy fails.
Divide & Conquer	No overlapping subproblems $\rightarrow$ D&C. Overlapping $\rightarrow$ DP.
Backtracking	DP is backtracking + memoization. DP avoids recomputation.
Shortest Path	DP is essentially shortest path on DAG of subproblems.
Memoization	The caching technique that makes DP efficient.

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Climbing Stairs	LeetCode 70	Fibonacci-like	Easy
House Robber	LeetCode 198	1D DP	Medium

Medium

Problem	Platform	Pattern	Difficulty
Longest Increasing Subsequence	LeetCode 300	LIS	Medium
Coin Change	LeetCode 322	Unbounded knapsack	Medium
Partition Equal Subset Sum	LeetCode 416	0/1 knapsack	Medium

Hard

Problem	Platform	Pattern	Difficulty
Edit Distance	LeetCode 72	String DP	Hard
Burst Balloons	LeetCode 312	Interval DP	Hard

Problem	Platform	Pattern	Difficulty
Distinct Subsequences	LeetCode 115	String DP	Hard

18. Interview Explanation

"DP is a technique for solving problems with overlapping subproblems and optimal substructure. I typically use bottom-up tabulation: I define the state, establish the recurrence relation, set base cases, and fill the table iteratively. For example, in 0/1 knapsack, $dp[i][w]$ represents the max value using first i items with capacity w , and the recurrence is either skip or take the current item. The time is $O(nW)$ and space can be optimized to $O(W)$ by using a 1D array with reverse iteration. I start by identifying whether a greedy solution exists, and if not, I look for overlapping subproblems. The key skill is defining the state correctly — I ask myself: what information do I need to remember from the past to make optimal future decisions?"

19. Revision Notes

- **Key idea:** Store subproblem results, avoid recomputation.
- **State:** $dp[i][j] = ??$ (what does it represent)
- **Transition:** How does $dp[i][j]$ depend on smaller states?
- **Base case:** $dp[0] = \text{base}$, $dp[i][0] = \text{base}$, etc.
- **Bottom-up vs Top-down:** Bottom-up = tabulation (iterative), Top-down = memoization (recursive).
- **0/1 Knapsack:** 1D optimized, iterate W backwards.
- **Unbounded Knapsack:** 1D optimized, iterate W forwards.
- **LCS:** Match $\rightarrow \text{diag}+1$, no match $\rightarrow \max(\text{up}, \text{left})$.
- **Common trap:** Wrong iteration direction, wrong base case, INF handling.

20. Final Cheat Sheet

DP — CHEAT SHEET	
WHEN TO USE:	Optimization, counting, with overlapping subproblems
STATE:	What $dp[i][j]$ represents
TRANSITION:	How state relates to smaller states
BASE CASE:	$dp[0]$, $dp[0][*]$, $dp[*][0]$
0/1 KNAPSACK:	for $w=W..wt$: $dp[w]=\max(dp[w], dp[w-wt]+val)$
UNBOUNDED:	for $w=wt..W$: $dp[w]=\max(...)$
LCS:	if match: $dp[i-1][j-1]+1$ else: $\max(dp[i-1][j], dp[i][j-1])$
LIS:	$O(n^2)$: $dp[i]=\max(dp[j]+1) \ j < i$ $O(n \log n)$: patience sort
COMPLEXITY:	$O(n^2)$ to $O(nW)$
TRAP:	Wrong direction, base case, INF
RELATED:	Greedy, Backtracking, D&C

14. TREE RECURSION

1. Overview

Tree recursion refers to recursive algorithms on tree data structures. Since trees are naturally recursive (each node is itself the root of a subtree), recursive solutions are elegant and concise. Common patterns include tree traversals, computing tree properties (height, diameter, max path sum), and building/modifying trees.

2. Intuition

Core idea: Solve the problem for the left subtree and right subtree recursively, then combine the results to solve for the current node. Trust that the recursive calls handle the subtrees correctly.

Analogy: You're a manager in a company. To find out the total headcount, you ask each of your direct reports to count their teams. They recursively ask their reports. Each person only needs to know their own count and sum up their team's counts. You assemble everyone's answers.

Step-by-step reasoning:

1. If the node is null, return the base case (0, null, true, etc.).
2. Recursively solve for the left child.
3. Recursively solve for the right child.
4. Combine the results to compute the answer for the current node.
5. Return the answer.

Why it works: Trees are recursively defined structures. The recursive algorithm mirrors the tree's structure, making it natural and correct by induction.

3. When to Use It

- **Tree traversals:** Preorder, inorder, postorder, level order.
- **Tree properties:** Height, depth, diameter, size.
- **Path-based problems:** Root-to-leaf paths, max path sum, path sum III.
- **Tree construction:** Build tree from traversals, serialize/deserialize.
- **Validation:** Check if tree is BST, balanced, symmetric.
- **Ancestor/LCA:** Lowest common ancestor.
- **Modification:** Invert tree, flatten tree, prune tree.

Common trigger phrases:

- "binary tree"
- "BST"
- "height" / "depth"
- "path"
- "subtree"
- "traversal"
- "diameter"
- "symmetric"

- "balanced"
- "lowest common ancestor"

4. When Not to Use It

- **Very deep tree:** Recursion may overflow the stack (depth > 10^5). Use iterative traversal with explicit stack.
- **Level-order processing:** BFS with queue is more natural.
- **Simple iteration:** If the problem is about walking down a single path (BST search), iterative while loop is simpler.
- **Performance-critical:** Recursion has function call overhead. Iterative can be faster.

5. Core Concepts

5.1 Node Structure

```
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

5.2 Three Traversal Orders

Preorder (Root → Left → Right): Process root before subtrees. Used for copying tree, prefix expression.

Inorder (Left → Root → Right): Process left subtree, then root, then right. For BST, gives sorted order.

Postorder (Left → Right → Root): Process root after subtrees. Used for deletion, computing size/diameter, postfix expression.

5.3 Divide and Conquer Pattern

```
Result solve(TreeNode* root) {
    if (!root) return base;
    Result left = solve(root->left);
    Result right = solve(root->right);
    return combine(root, left, right);
}
```

5.4 Global Variable Pattern

Sometimes we need to track information across recursive calls (e.g., max diameter, previous node in BST validation).

```

int maxDiameter = 0;
int height(TreeNode* root) {
    if (!root) return 0;
    int left = height(root->left);
    int right = height(root->right);
    maxDiameter = max(maxDiameter, left + right);
    return 1 + max(left, right);
}

```

6. Step-by-Step Algorithm

Tree Height / Max Depth:

1. If root is null: return 0.
2. Recursively compute height of left subtree: `left = height(root->left)`.
3. Recursively compute height of right subtree: `right = height(root->right)`.
4. Return `1 + max(left, right)`.

Check if Tree is BST:

1. Define helper `isValidBST(node, minVal, maxVal)`.
2. If node is null: return true.
3. If `node->val <= minVal || node->val >= maxVal`: return false.
4. Recursively check left subtree with range `(minVal, node->val)`.
5. Recursively check right subtree with range `(node->val, maxVal)`.

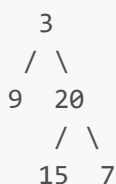
Lowest Common Ancestor (LCA):

1. If root is null or root is p or root is q: return root.
2. Recursively search left and right subtrees.
3. If both left and right return non-null: root is LCA.
4. If only one side returns non-null: that side has both p and q, return that result.

7. Dry Run

Problem: Max Depth of Binary Tree

Tree:



Node	Left Height	Right Height	Result = 1 + max(L,R)
9 (leaf)	0	0	1
15 (leaf)	0	0	1
7 (leaf)	0	0	1
20	1 (15)	1 (7)	2
3	1 (9)	2 (20)	3

Max Depth = 3

Problem: LCA of nodes 15 and 7

Node	Left Result	Right Result	Action
9	null	null	Return null
15	null	null	Return 15 (is target)
7	null	null	Return 7 (is target)
20	15 (found!)	7 (found!)	Return 20 (LCA!)
3	null	20	Return 20

LCA = 20

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class TreeRecursion {
public:
    // =====
    // 1. Max Depth (Height)
    // =====
    int maxDepth(TreeNode* root) {
        if (!root) return 0;
        return 1 + max(maxDepth(root->left), maxDepth(root->right));
    }

    // =====

```

```

// 2. Validate BST
// =====
bool isValidBST(TreeNode* root) {
    return isValidBSTHelper(root, LONG_MIN, LONG_MAX);
}

bool isValidBSTHelper(TreeNode* node, long minVal, long maxVal) {
    if (!node) return true;
    if (node->val <= minVal || node->val >= maxVal) return false;
    return isValidBSTHelper(node->left, minVal, node->val) &&
        isValidBSTHelper(node->right, node->val, maxVal);
}

// =====
// 3. Diameter of Binary Tree
// =====
int diameterOfBinaryTree(TreeNode* root) {
    int diameter = 0;
    heightWithDiameter(root, diameter);
    return diameter;
}

int heightWithDiameter(TreeNode* node, int& diameter) {
    if (!node) return 0;
    int left = heightWithDiameter(node->left, diameter);
    int right = heightWithDiameter(node->right, diameter);
    diameter = max(diameter, left + right);
    return 1 + max(left, right);
}

// =====
// 4. Lowest Common Ancestor
// =====
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;

    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);

    if (left && right) return root; // p and q in different subtrees
    return left ? left : right; // both in same subtree
}

// =====
// 5. Binary Tree Maximum Path Sum
// =====
int maxPathSum(TreeNode* root) {
    int maxSum = INT_MIN;
    maxGain(root, maxSum);
    return maxSum;
}

int maxGain(TreeNode* node, int& maxSum) {
    if (!node) return 0;

```

```

    int leftGain = max(0, maxGain(node->left, maxSum));
    int rightGain = max(0, maxGain(node->right, maxSum));

    // Path that goes through this node
    int currentPath = node->val + leftGain + rightGain;
    maxSum = max(maxSum, currentPath);

    // Return max path that can be extended to parent
    return node->val + max(leftGain, rightGain);
}

// =====
// 6. Invert Binary Tree
// =====
TreeNode* invertTree(TreeNode* root) {
    if (!root) return nullptr;
    swap(root->left, root->right);
    invertTree(root->left);
    invertTree(root->right);
    return root;
}

// =====
// 7. Flatten Binary Tree to Linked List
// =====
void flatten(TreeNode* root) {
    if (!root) return;

    flatten(root->left);
    flatten(root->right);

    // Store right subtree
    TreeNode* rightSubtree = root->right;

    // Move left subtree to right
    root->right = root->left;
    root->left = nullptr;

    // Find the end of the new right subtree
    TreeNode* curr = root;
    while (curr->right) curr = curr->right;

    // Attach the original right subtree
    curr->right = rightSubtree;
}

};

// Helper to build a simple tree for testing
TreeNode* buildExampleTree() {
    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(9);
    root->right = new TreeNode(20);
    root->right->left = new TreeNode(15);

```

```

    root->right->right = new TreeNode(7);
    return root;
}

// Example usage
int main() {
    TreeRecursion tr;

    TreeNode* root = buildExampleTree();

    cout << "Max Depth: " << tr.maxDepth(root) << "\n"; // 3
    cout << "Is Valid BST: " << tr.isValidBST(root) << "\n"; // 1
    cout << "Diameter: " << tr.diameterOfBinaryTree(root) << "\n"; // 3

    TreeNode* p = root->right->left; // 15
    TreeNode* q = root->right->right; // 7
    TreeNode* lca = tr.lowestCommonAncestor(root, p, q);
    cout << "LCA of 15 and 7: " << lca->val << "\n"; // 20

    cout << "Max Path Sum: " << tr.maxPathSum(root) << "\n";

    TreeNode* inverted = tr.invertTree(root);
    cout << "Inverted root left->val: " << inverted->left->val << "\n";

    return 0;
}

```

9. Python Implementation

```

from typing import Optional, List

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class TreeRecursion:
    # =====
    # 1. Max Depth
    # =====
    def max_depth(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        return 1 + max(self.max_depth(root.left), self.max_depth(root.right))

    # =====
    # 2. Validate BST
    # =====
    def is_valid_bst(self, root: Optional[TreeNode]) -> bool:
        def helper(node: Optional[TreeNode], min_val: float, max_val: float) ->

```

```

bool:
    if not node:
        return True
    if node.val <= min_val or node.val >= max_val:
        return False
    return helper(node.left, min_val, node.val) and \
        helper(node.right, node.val, max_val)
return helper(root, float('-inf'), float('inf'))

# =====
# 3. Diameter
# =====
def diameter_of_binary_tree(self, root: Optional[TreeNode]) -> int:
    diameter = 0

    def height(node: Optional[TreeNode]) -> int:
        nonlocal diameter
        if not node:
            return 0
        left = height(node.left)
        right = height(node.right)
        diameter = max(diameter, left + right)
        return 1 + max(left, right)

    height(root)
    return diameter

# =====
# 4. Lowest Common Ancestor
# =====
def lowest_common_ancestor(self, root: Optional[TreeNode],
                           p: Optional[TreeNode],
                           q: Optional[TreeNode]) -> Optional[TreeNode]:
    if not root or root == p or root == q:
        return root

    left = self.lowest_common_ancestor(root.left, p, q)
    right = self.lowest_common_ancestor(root.right, p, q)

    if left and right:
        return root
    return left or right

# =====
# 5. Max Path Sum
# =====
def max_path_sum(self, root: Optional[TreeNode]) -> int:
    max_sum = float('-inf')

    def max_gain(node: Optional[TreeNode]) -> int:
        nonlocal max_sum
        if not node:
            return 0

```

```

        left_gain = max(0, max_gain(node.left))
        right_gain = max(0, max_gain(node.right))

        current_path = node.val + left_gain + right_gain
        max_sum = max(max_sum, current_path)

        return node.val + max(left_gain, right_gain)

    max_gain(root)
    return max_sum

# Example usage
if __name__ == "__main__":
    tr = TreeRecursion()

    # Build tree
    root = TreeNode(3)
    root.left = TreeNode(9)
    root.right = TreeNode(20)
    root.right.left = TreeNode(15)
    root.right.right = TreeNode(7)

    print(f"Max Depth: {tr.max_depth(root)}") # 3
    print(f"Is BST: {tr.is_valid_bst(root)}") # True
    print(f"Diameter: {tr.diameter_of_binary_tree(root)}") # 3

    lca = tr.lowest_common_ancestor(root, root.right.left, root.right.right)
    print(f"LCA: {lca.val}") # 20

    print(f"Max Path Sum: {tr.max_path_sum(root)}")

```

10. Code Explanation

Max Depth:

- Base case: null node has depth 0.
- Recursive case: depth = 1 + max(depth of left, depth of right).
- The recursion goes as deep as the tree is tall.

Validate BST:

- Each node has a valid range (min, max).
- Left child must be within (min, node.val).
- Right child must be within (node.val, max).
- Use long/min to handle edge values (INT_MIN, INT_MAX).

Diameter:

- For each node, the diameter through that node = left height + right height.
- Global variable tracks the max across all nodes.
- The function returns height for use by parent.

LCA:

- If root is null or matches p or q, return root.
- Recursively search left and right.
- If both sides returned non-null, root is the LCA.
- If only one side returned non-null, propagate that side up.

Max Path Sum:

- For each node, the max path through it = $\text{node.val} + \text{leftGain} + \text{rightGain}$.
- Gains are $\max(0, \dots)$ because we can choose not to include negative contributions.
- The returned value is $\max(\text{node.val} + \text{leftGain}, \text{node.val} + \text{rightGain})$ — only one side can extend to parent.

11. Complexity Analysis

Problem	Time	Space (Recursion Stack)
Max Depth	$O(n)$	$O(h)$ worst $O(n)$
Validate BST	$O(n)$	$O(h)$
Diameter	$O(n)$	$O(h)$
LCA	$O(n)$	$O(h)$
Max Path Sum	$O(n)$	$O(h)$
Invert Tree	$O(n)$	$O(h)$
Flatten	$O(n)$	$O(h)$

n = number of nodes, h = height of tree.

- Best case (balanced): $h = \log n$
- Worst case (skewed): $h = n$

12. Common Patterns

Pattern 1: Simple Recursion (Divide & Conquer)

Identify: Problem can be solved by combining left and right results. **Approach:** Postorder recursion.

Examples: Max depth, count nodes, sum of all values.

Pattern 2: Global Variable Pattern

Identify: Need to track a value across all nodes (max, count, prev). **Approach:** Use a reference/pointer variable updated by recursive function. **Examples:** Diameter, max path sum, count good nodes.

Pattern 3: Range Validation

Identify: Validate BST, check if tree satisfies property. **Approach:** Pass range parameters down. **Example:** Validate BST.

Pattern 4: LCA Pattern

Identify: Find common ancestor of two nodes. **Approach:** Recursively search left/right, return based on results. **Example:** LCA of binary tree.

Pattern 5: Construct Tree from Traversals

Identify: Build tree from inorder + preorder/postorder. **Approach:** Use preorder to find root, split inorder, recurse. **Example:** LeetCode 105 — Construct Binary Tree from Preorder and Inorder.

13. Common Mistakes

- **Forgetting base case:** Always check for null root first.
- **Stack overflow:** Very deep trees (10^5 nodes in a line) can overflow recursion stack.
- **Not using long for BST validation:** INT_MIN/INT_MAX are valid node values, preventing proper range checks.
- **Wrong traversal order:** Postorder for computing child-dependent values, preorder for constructing.
- **Not handling negative values in max path sum:** Gains can be negative; we clip them to 0.
- **Incorrect LCA logic for BST:** For BST, we can use value comparisons to narrow search.

14. Edge Cases

Case	Expected Behavior
Empty tree (null root)	Return 0, true, null, etc. based on problem
Single node	Height = 1, diameter = 0, path sum = node.val
Skewed tree (linked list)	Recursion depth = n
All negative values	Max path sum = max negative value (not 0)
p or q not in tree	LCA returns null or the found node
p == q	LCA = p (or q)

15. Variations

Iterative Tree Traversal

Use explicit stack for preorder, inorder, postorder to avoid recursion overhead.

Morris Traversal ($O(1)$ Space)

Threaded binary tree traversal. Modifies tree temporarily to achieve $O(1)$ space.

Tree Recursion with Memoization

For problems where computing subtree values repeatedly, memoization can help.

Sum Root to Leaf Numbers

Each path from root to leaf forms a number. Sum all such numbers.

16. Related Algorithms/Data Structures

Structure	Connection
Binary Search Tree	Tree recursion on BST has additional ordering properties.
Segment Tree	Tree recursion used for range queries and updates.
Trie	Tree recursion for prefix-based operations.
Graph DFS	Tree recursion is a special case of DFS on trees (no cycles).

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Maximum Depth of Binary Tree	LeetCode 104	Simple recursion	Easy
Invert Binary Tree	LeetCode 226	Tree modification	Easy

Medium

Problem	Platform	Pattern	Difficulty
Validate Binary Search Tree	LeetCode 98	Range validation	Medium
Diameter of Binary Tree	LeetCode 543	Global variable	Easy
Lowest Common Ancestor	LeetCode 236	LCA pattern	Medium

Hard

Problem	Platform	Pattern	Difficulty
Binary Tree Maximum Path Sum	LeetCode 124	Global variable + gains	Hard
Serialize and Deserialize Binary Tree	LeetCode 297	Tree construction	Hard
Binary Tree Cameras	LeetCode 968	DP on tree	Hard

18. Interview Explanation

"Tree recursion is a natural fit for tree problems because trees are recursively defined. I use a divide-and-conquer approach: recursively solve for the left and right subtrees, then combine the results for the current node. The base case is always null. For example, to find the max depth, I return $1 + \max(\text{depth of left}, \text{depth of right})$. For validating BST, I pass down a valid range. For problems like diameter or max path sum, I use a global variable to track the answer while the recursive function returns the value needed by the parent. The time complexity is $O(n)$ and space is $O(h)$ for the recursion stack."

19. Revision Notes

- **Core pattern:** Divide (left, right) → Conquer (combine) → Return
- **Base case:** `if (!root) return ...`
- **Traversals:** Preorder (root first), Inorder (root middle), Postorder (root last)
- **BST validation:** Range parameters (min, max)
- **Diameter:** `diameter = max(diameter, leftHeight + rightHeight)`
- **LCA:** If left and right both found → root is LCA
- **Max path sum:** `gain = max(0, childGain)`, `path = node.val + leftGain + rightGain`
- **Complexity:** O(n) time, O(h) space
- **Common trap:** Stack overflow for deep trees, wrong range in BST

20. Final Cheat Sheet

TREE RECURSION – CHEAT SHEET	
WHEN TO USE:	Any binary tree problem
PATTERN:	<pre> if(!root) return base; left = f(root->left); right = f(root->right); return combine(root, left, right); </pre>
MAX DEPTH:	<code>1 + max(depth(left), depth(right))</code>
BST:	<code>helper(node, min, max)</code>
DIAMETER:	<code>max(d, leftH + rightH)</code>
LCA:	<code>if(left && right) return root</code>
PATH SUM:	<pre> gain = max(0, childGain) maxSum = max(maxSum, val+L+R) return val + max(L, R) </pre>
COMPLEXITY:	O(n) time, O(h) space
TRAVERSAL:	Pre/In/Post order
TRAP:	Stack overflow, base case, range
RELATED:	DFS, BST, Segment Tree, Trie

15. GRAPH TRAVERSAL

1. Overview

Graph traversal algorithms systematically visit all vertices and edges of a graph. The two fundamental traversals are **BFS** (Breadth-First Search) and **DFS** (Depth-First Search). This section covers their application to various graph problems beyond simple traversal.

2. Intuition

Core idea: Graph traversal is about exploring a graph systematically. BFS explores like ripples in water (level by level), while DFS explores like a maze explorer (going deep and backtracking). The choice depends on the problem.

Analogy (BFS): Broadcasting a message through a phone tree. You call all your contacts first (level 1). They call their contacts (level 2). The message spreads in waves.

Analogy (DFS): Exploring a cave system. You pick a tunnel and go as far as you can. When you hit a dead end, you backtrack and try another tunnel.

3-20. Reference BFS/DFS Sections

Note: For detailed explanations of BFS and DFS (Code Explanation, Complexity, Patterns, Mistakes, Edge Cases, etc.), refer to the dedicated [BFS Section](#) and [DFS Section](#) above. This section covers advanced graph traversal algorithms: topological sort, SCC, bipartite check, and cycle detection.

Additional Algorithms

Kahn's Algorithm (BFS Topological Sort):

- Compute in-degree for each node.
- Push nodes with indegree 0 to queue.
- Pop, add to result, decrement neighbors' indegree.
- If result size $\neq V$, graph has a cycle.

Kosaraju's SCC:

- DFS on original graph, store finish order.
- Reverse all edges.
- DFS on reversed graph in reverse finish order.
- Each DFS tree = one SCC.

Complexity

Algorithm	Time	Space
Kahn's Topo	$O(V+E)$	$O(V)$
Kosaraju SCC	$O(V+E)$	$O(V+E)$
Bipartite Check	$O(V+E)$	$O(V)$
Cycle (Directed)	$O(V+E)$	$O(V)$

Practice Problems

Problem	Platform	Algorithm	Difficulty
Course Schedule II	LeetCode 210	Topological sort	Medium
Find Eventual Safe States	LeetCode 802	Cycle detection	Medium
Is Graph Bipartite?	LeetCode 785	BFS 2-coloring	Medium

16. UNION-FIND (DISJOINT SET UNION — DSU)

1. Overview

Union-Find tracks a set of elements partitioned into disjoint subsets. It supports **Find** (which set?) and **Union** (merge two sets) in nearly $O(1)$ amortized time.

2. Intuition

Core idea: Each element starts in its own group. When two elements connect, their groups merge. We can efficiently answer "are these two elements connected?" at any point.

3. When to Use It

- Dynamic connectivity
- Cycle detection in undirected graphs
- Kruskal's MST
- Redundant connections
- Number of islands II (dynamic additions)

4. Core Concepts

Path Compression: During **find**, make node point directly to root. **Union by Rank:** Attach smaller tree under larger tree.

5. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class UnionFind {
    vector<int> parent, rank;
    int components;
public:
    UnionFind(int n) : parent(n, 0), rank(n, 0), components(n) {
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    bool unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;
        if (rank[rx] < rank[ry]) swap(rx, ry);
        parent[ry] = rx;
        if (rank[rx] == rank[ry]) rank[rx]++;
        components--;
        return true;
    }
}
```

```
bool connected(int x, int y) { return find(x) == find(y); }
int getComponents() { return components; }
};
```

6. Python Implementation

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.components = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def unite(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        self.components -= 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)
```

7. Complexity

Operation	Time	Space
Find	$O(\alpha(n)) \approx O(1)$	$O(1)$
Union	$O(\alpha(n)) \approx O(1)$	$O(1)$

8. Common Patterns

- **Redundant Connection:** Process edges, return first where `unite` returns false.
- **Kruskal's MST:** Sort edges by weight, `unite` if no cycle, add to MST weight.
- **Grid DSU:** Map cell (r, c) to $id = r * cols + c$.

9. Practice Problems

Problem	Platform	Pattern	Difficulty
Redundant Connection	LeetCode 684	Cycle detection	Medium
Accounts Merge	LeetCode 721	Email merging	Medium
Min Cost to Connect All Points	LeetCode 1584	Kruskal's MST	Medium

10. Final Cheat Sheet

UNION-FIND (DSU) – CHEAT SHEET	
WHEN TO USE:	Dynamic connectivity, cycle detect in undirected graph, Kruskal's MST
FIND:	<code>if(parent[x]!=x) parent[x]=find(..)</code>
UNITE:	<code>rx=find(x), ry=find(y)</code> <code>if(rx==ry) return false</code> <code>if(rank[rx]<rank[ry]) swap</code> <code>parent[ry]=rx; if(rank same) rank++</code> <code>components--; return true</code>
COMPLEXITY:	$O(\alpha(n)) \approx O(1)$ per operation
GRID DSU:	<code>id = r * cols + c</code>
TRAP:	No path compression, forgetting components--, wrong initialization
RELATED:	BFS/DFS, Kruskal, Graph

End of Document — Must-Know Patterns for Placements & Competitive Programming (Part 1)